

ECE 647: Lecture 23

Sometimes we can explicitly solve for the optimal solution from the optimality conditions. For example, consider the linear regression problem:

$$\begin{aligned} \min \|Ax - b\|_2^2 \\ \Rightarrow A^T Ax = A^T b \\ \Rightarrow x^* = (A^T A)^{-1} A^T b \end{aligned}$$

At other times, a closed form solution may not be possible. In such cases, we can use iterative, numerical algorithms to solve the optimization problem.

Numerical Algorithms

The common procedure of a numerical algorithm is as follows:

1. Start with an initial guess x_0 .
2. Iteratively generate new estimates by

$$x_k = T(x_{k-1}), \quad k = 1, 2, \dots$$

where T is a function that generates the new estimate from the previous estimate.

3. Hopefully, as $k \rightarrow \infty$, $x_k \rightarrow x^*$. That is, the sequence $\{x_k\}$ converges to the optimal solution.

Q When will the sequence $\{x_k\}$ converge to the optimal solution?

A Often we prove the convergence by showing that the quality of the estimate x_k improves with each iteration.

- (a) For example, we can show that the distance between x_k and x^* decreases with each iteration, e.g.,

$$\begin{aligned} \|x_k - x^*\| &\leq \alpha \|x_{k-1} - x^*\|, \quad \alpha < 1 \\ \text{or } \|x_k - x^*\| &\leq \|x_{k-1} - x^*\| - \beta, \quad \beta > 0 \end{aligned}$$

- (b) We can also show that the objective function value decreases with each iteration, e.g.,

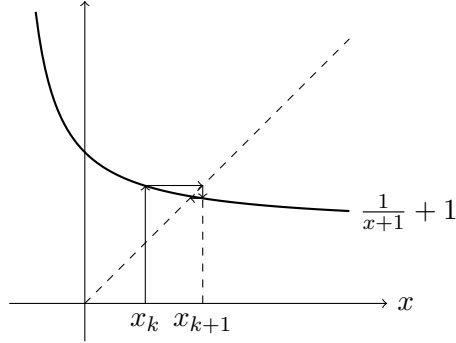
$$f_o(x_k) \leq f_o(x_{k-1}) - \gamma, \quad \gamma > 0.$$

A trivial example

- How to compute $\sqrt{2}$ using only $+$, $-$, $\times$, $\div$ operations?

$$x = \sqrt{2} \Leftrightarrow x^2 - 1 = 1 \Leftrightarrow x = \frac{1}{x+1} + 1$$

- The iterative update is thus given by $x_k = \frac{1}{x_{k-1}+1} + 1$.



- For any two points x, y , let

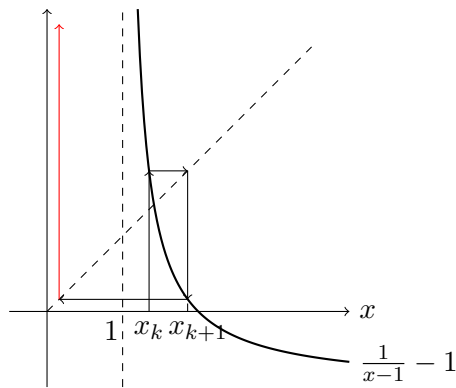
$$\begin{aligned} T(x) &= \frac{1}{x+1} + 1, \quad T(y) = \frac{1}{y+1} + 1, \\ T(x) - T(y) &= \frac{1}{x+1} - \frac{1}{y+1} = -\frac{x-y}{(x+1)(y+1)}, \\ \Rightarrow \|T(x) - T(y)\| &\leq \frac{\|x-y\|}{4} \quad \text{assuming that } x, y \geq 1. \quad (*) \end{aligned}$$

Note that $\sqrt{2}$ is a fixed point of the update rule, i.e., $T(\sqrt{2}) = \sqrt{2}$. Based on the inequality (*), the distance $\|x_k - \sqrt{2}\|$ is cut by 1/4 at each iteration. As $k \rightarrow \infty$, $x_k \rightarrow \sqrt{2}$.

- The inequality (*) describes a *contraction mapping*:
 - The distance between $T(x)$ and $T(y)$ is smaller than the distance between x and y .

$$\|T(x) - T(y)\| \leq C\|x - y\|, \quad 0 \leq C \leq 1.$$

- When $C < 1$, the contraction mapping is strict, we immediately have
 - ① There is a unique fixed point x^* with $T(x^*) = x^*$.
 - ② The sequence $x_{k+1} = T(x_k)$ converges “geometrically” to x^* .
- However, not all iterative algorithms are contraction mappings. For example, the update rule $x_k = \frac{1}{x_{k-1}-1} - 1$, which is also equivalent to $x = \sqrt{2}$, does not converge to a fixed point.



Unconstrained Optimization

- $\min f(x)$ f is convex
- Optimality condition
 - If f is differentiable

$$\nabla f(x^*) = 0$$
 - If f is not differentiable

$$f'(x^*; x - x^*) \geq 0 \quad \forall x$$

i.e., the directional derivative is positive in all directions.

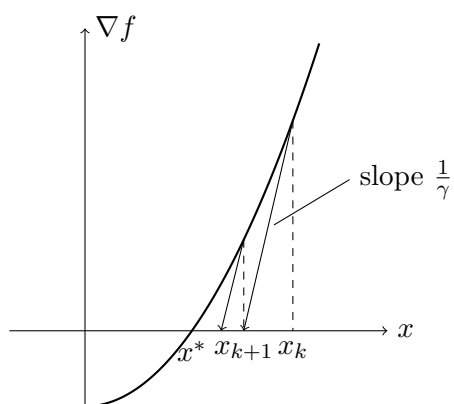
-
- Assume that $f(\cdot)$ is differentiable.

- Consider the iteration of the type

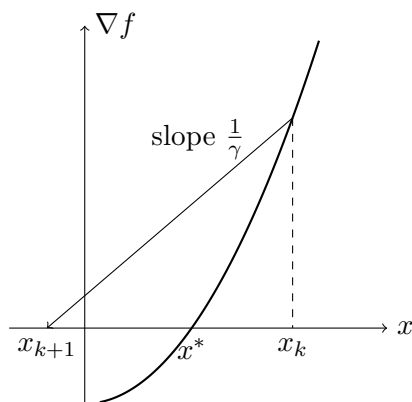
$$x^{(t+1)} = x^{(t)} - \gamma \nabla f(x^{(t)}) \triangleq T(x^{(t)})$$

↑ gradient

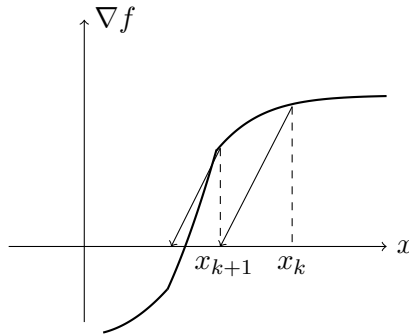
- Note that x^* is a fixed point of the iteration/mapping: if $x(t) = x^*$, then $x(t+1) = x^*$.
- The smaller γ is, the smaller each update is. Illustrations on how the update rule works:



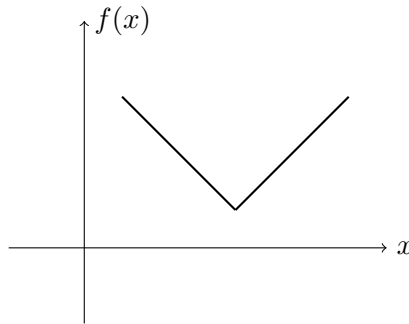
$x_k \rightarrow x^*$ if γ is small enough



May not converge if γ is too large



May be more difficult when ∇f is not “smooth”. This happens when f has sharp corners.



Proof of convergence

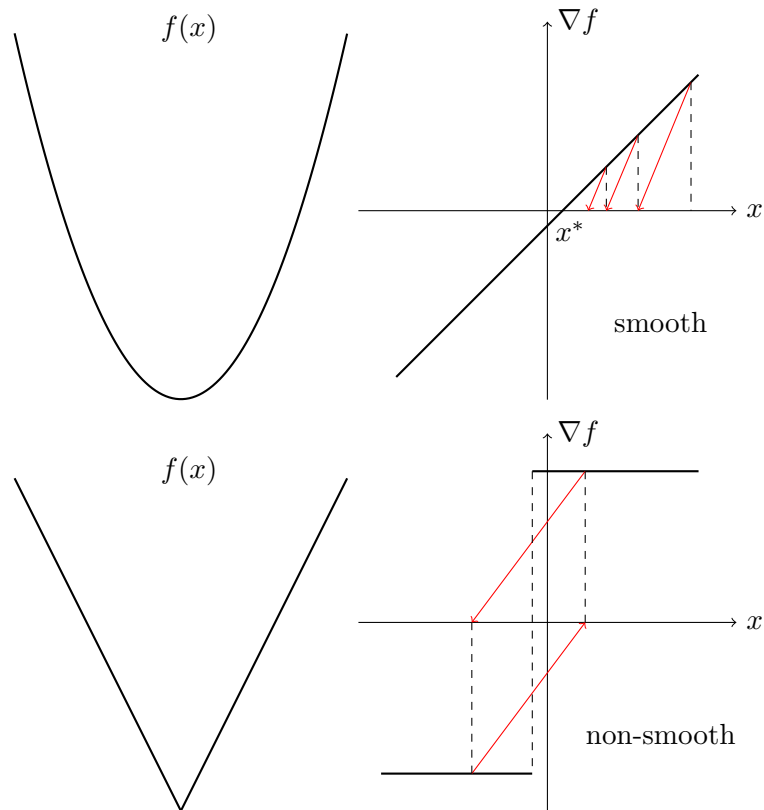
- How can we study the convergence of the gradient algorithm?
- We will usually encounter two types of conditions on the gradient $\nabla f(x)$

① Smoothness

- $\nabla f(x)$ does not abruptly change as x change

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2,$$

where L is the Lipschitz constant.



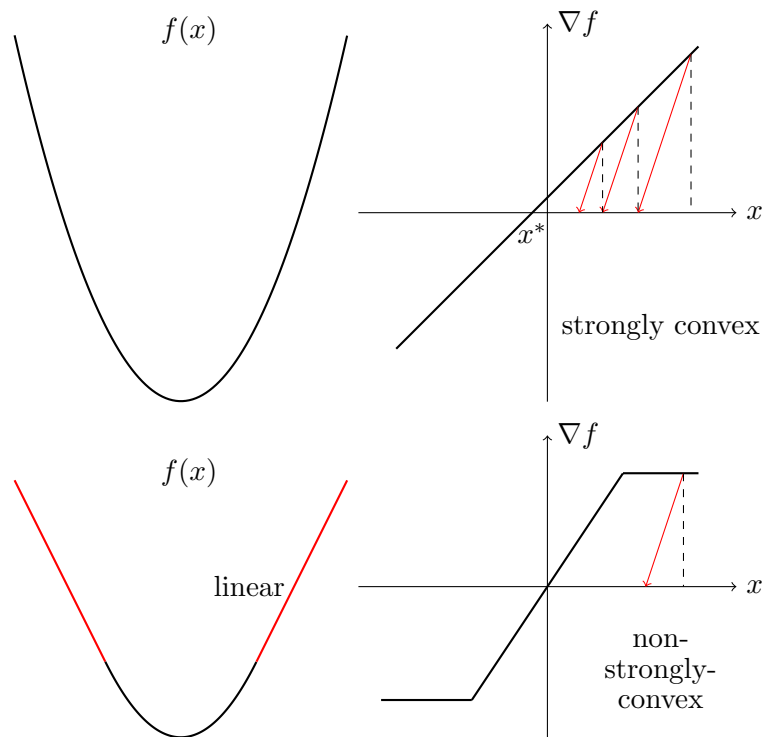
- For non-smooth func, even when you are already in a neighborhood of x^* , it would be difficult for the gradient algorithm to converge exactly to x^* .
- Smoothness implies convergence, provided that the stepsize γ is sufficiently small.

② Strong convexity

- $f(x)$ increases quickly as x lies away from x^*

$$\|\nabla f(x) - \nabla f(y)\|_2 \geq \alpha \|x - y\|_2$$

- If the function is not strongly convex, then starting from $x^{(0)}$ that is far away from x^* , the improvement of the gradient algorithm will be slow.



- Instead, strong-convexity & smoothness combined implies geometric descent

$$\|x^{(t+1)} - x^*\|_2 \leq \rho \|x^{(t)} - x^*\|_2, \quad \rho < 1$$

- This is usually referred to as "linear convergence" in the optimization literature.
 - In contrast to "quadratic convergence" by Newton's algorithm.

Two proof techniques:

- ① Show decrease of $f(x^{(t+1)})$: More intuitive but less powerful results.
- ② Show decrease of $\|x^{(t)} - x^*\|$.

First Approach: Decrease of f

Lemma: Assume that function f is continuously differentiable & there exists a constant L such that

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2$$

Then

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|_2^2$$

Note: The Lipschitz condition bounds how smooth $\nabla f(\cdot)$ is.

Proof: Let $g(t) = f(x + t(y - x))$. Then $g'(t) = \nabla f(x + t(y - x))^T(y - x)$.

$$\begin{aligned} f(y) &= f(x + (y - x)) = g(1) \\ &= f(x) + \int_0^1 \nabla f(x + t(y - x))^T \cdot (y - x) dt \\ &= f(x) + \nabla f(x)^T(y - x) \\ &\quad + \int_0^1 [\nabla f(x + t(y - x)) - \nabla f(x)]^T(y - x) dt \\ &\leq f(x) + \nabla f(x)^T(y - x) \\ &\quad + \int_0^1 \|\nabla f(x + t(y - x)) - \nabla f(x)\|_2 \|y - x\|_2 dt \\ &\leq f(x) + \nabla f(x)^T(y - x) \\ &\quad + \int_0^1 Lt\|y - x\|_2^2 dt \\ &= f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|_2^2. \end{aligned}$$

Declaration: This document was prepared with the assistance of Google Gemini 2.0 Flash Thinking. This LLM is very powerful in OCR and can be used to convert handwritten notes and math equations into digital forms. It is available in Google AI Studio (<https://aistudio.google.com/>) for free!